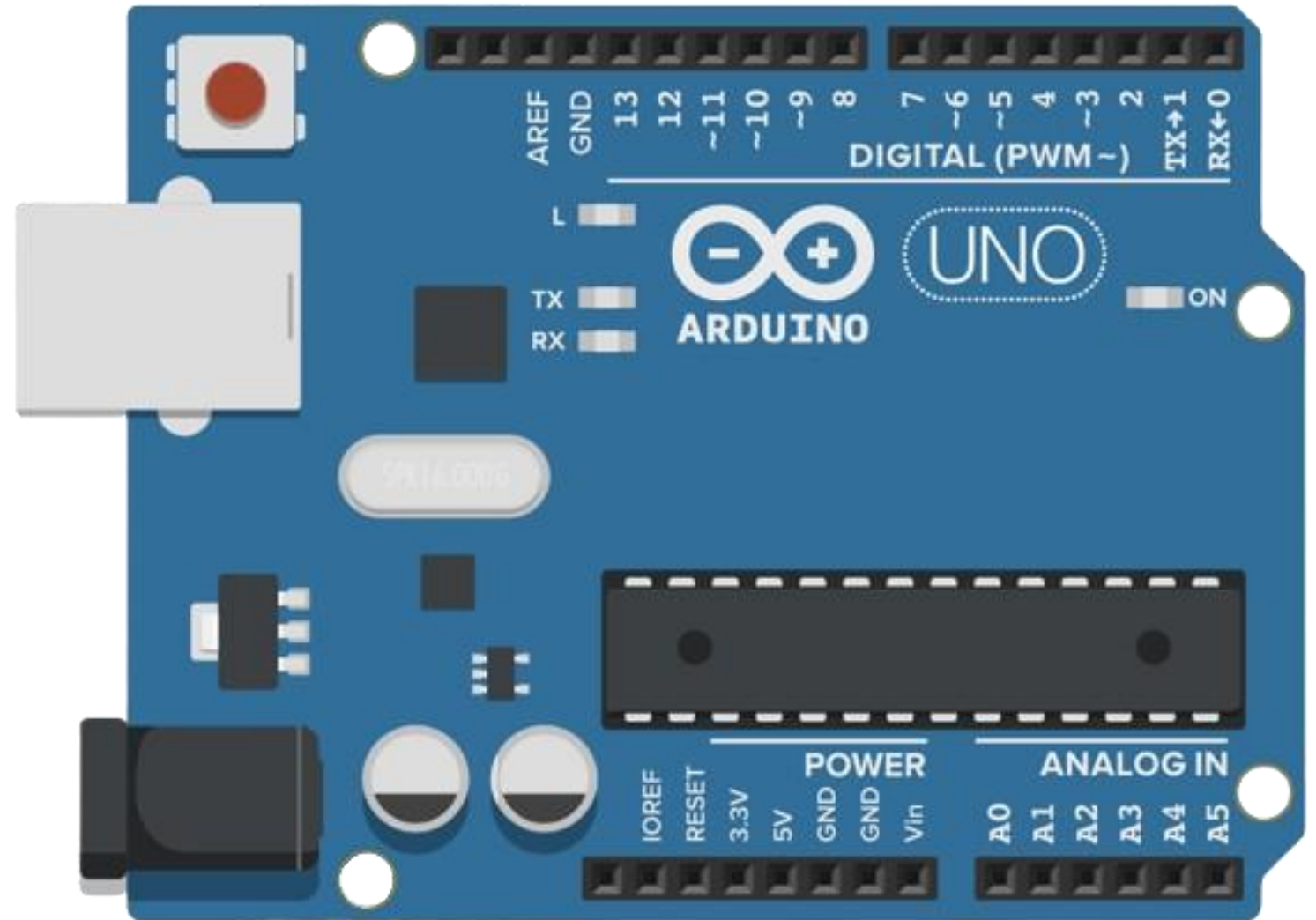
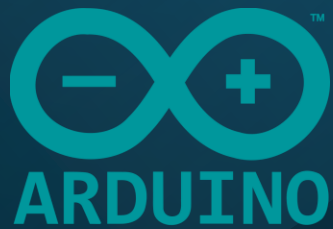


Gianmarco Melchiori &
Luca Palmeston

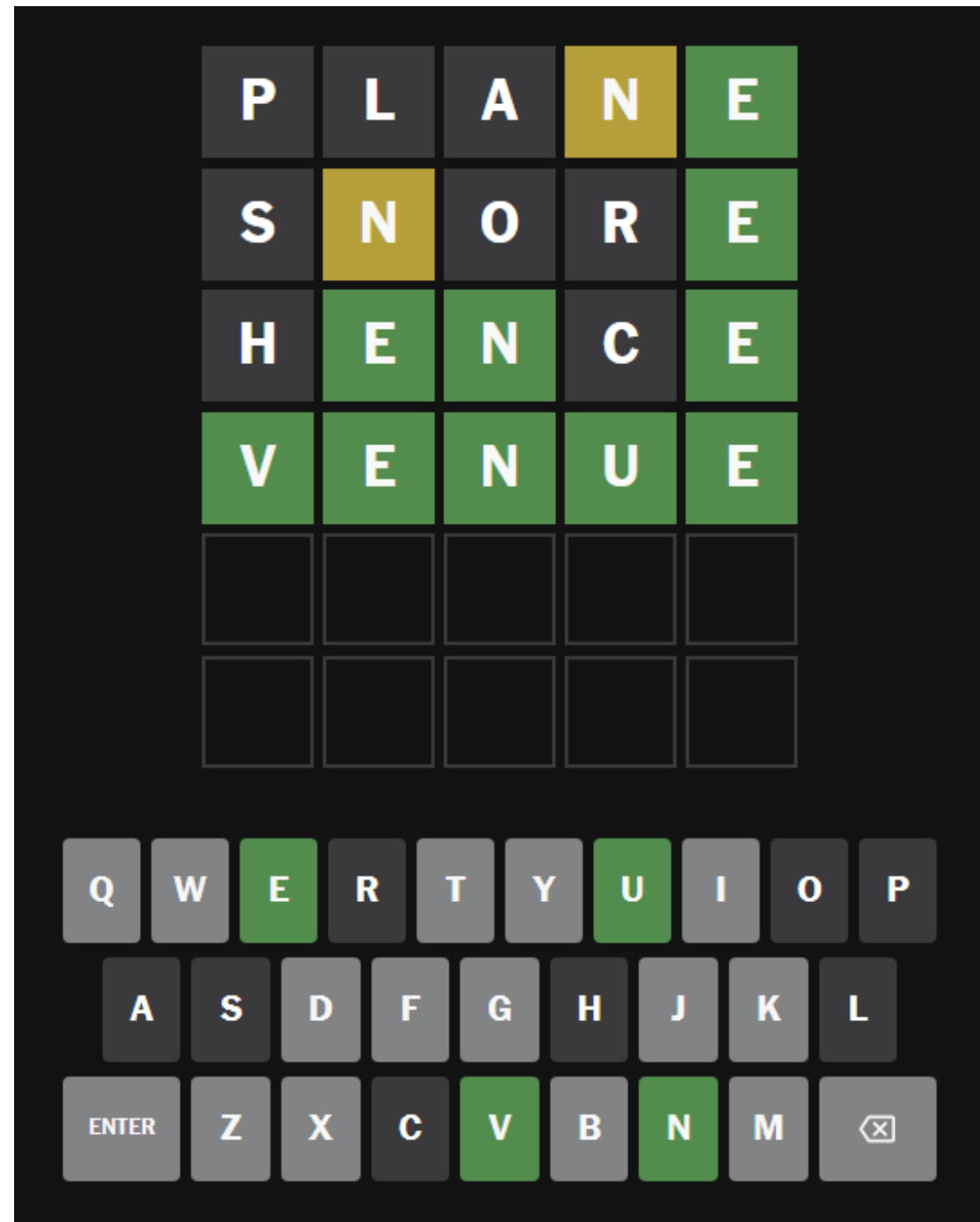
Prototipizzazione con Arduino



19 novembre 2025



Il gioco a cui
ci siamo
ispirati:
Wordle





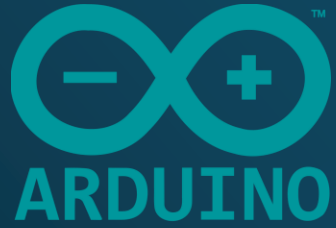
Il nostro
gioco

WORDLE ESP32+ARDUINO

🎉 HAI VINTO! 🎉

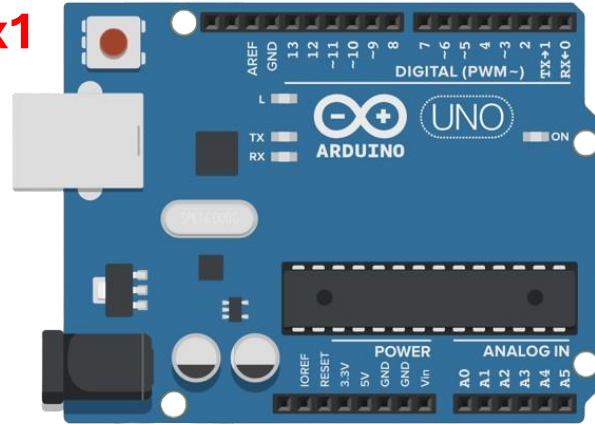
6	3	1
6	6	3
6	5	3

Nuova Partita

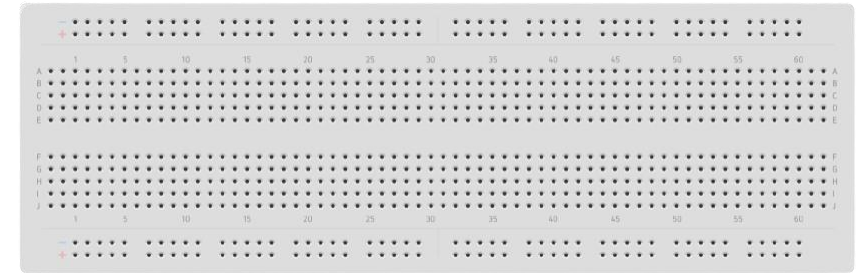


Componenti Hardware utilizzati

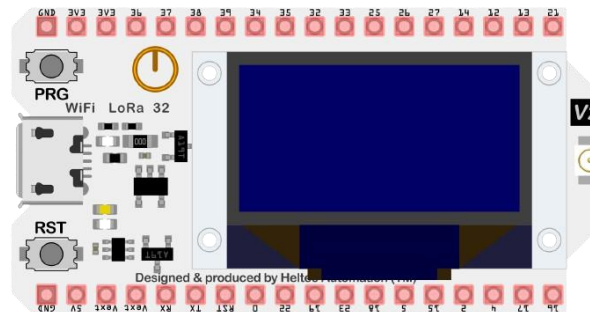
x1



x1



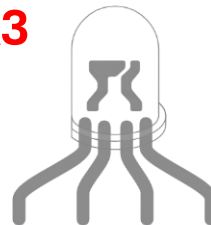
x1



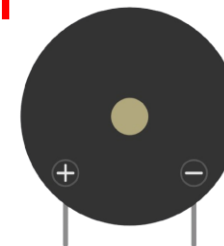
x7



x3

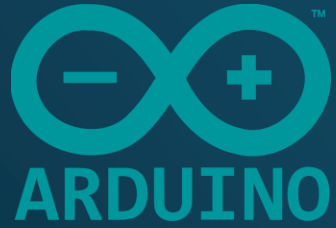


x1

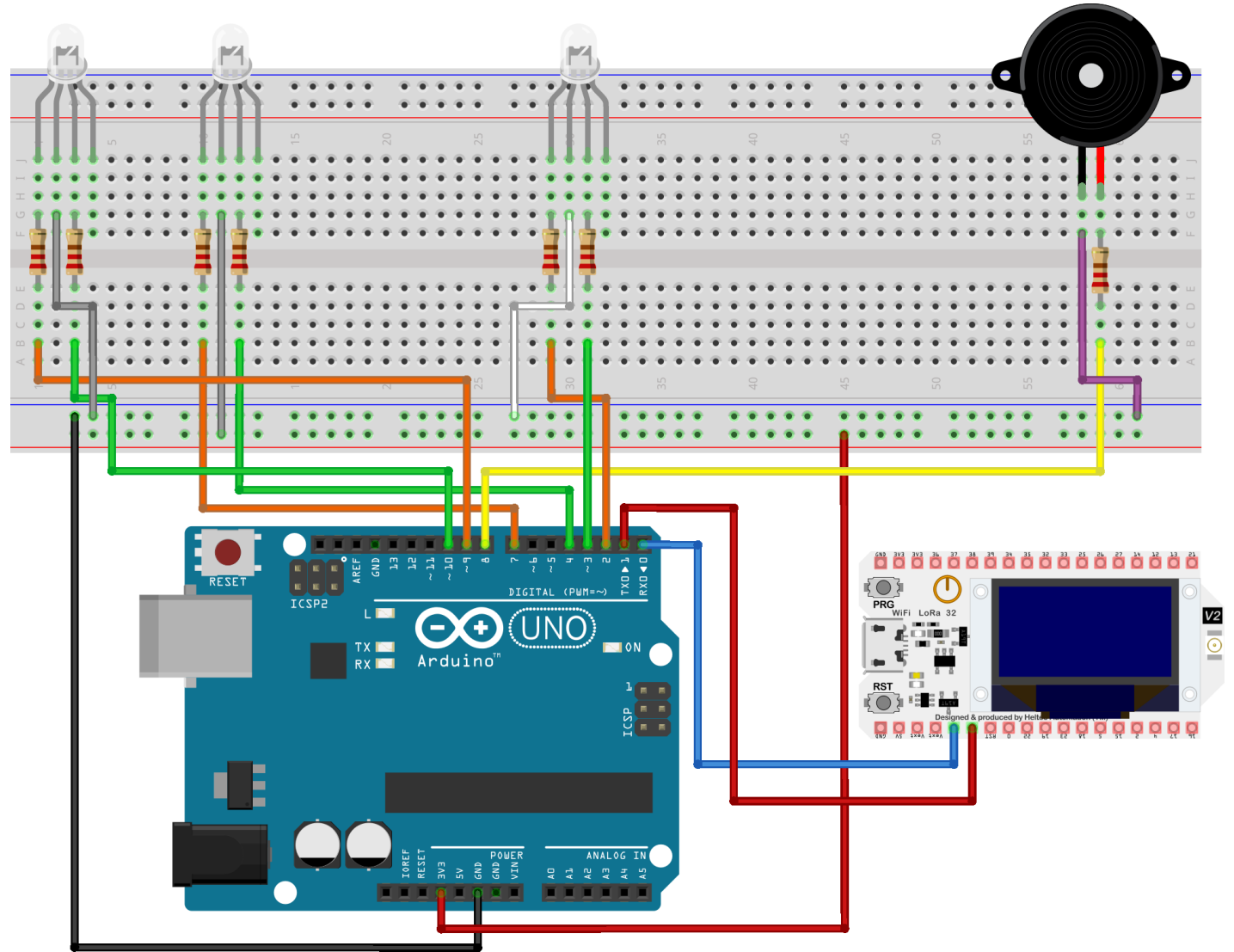


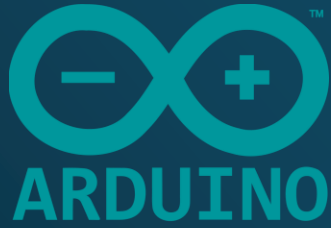
x15





Lo schema



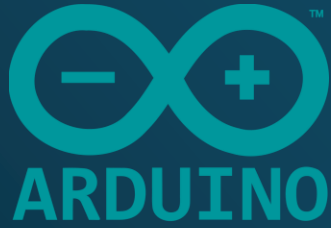


Il codice dell'Arduino

Nel codice della scheda Arduino UNO sono presenti sette funzioni:

- void setup()

```
25 void setup() {  
26     Serial.begin(9600);  
27  
28     // imposto tutti i pin di tutti i led in modalità OUTPUT  
29     pinMode(red1, OUTPUT);  
30     pinMode(green1, OUTPUT);  
31     // pinMode(blue1, OUTPUT);  
32     pinMode(red2, OUTPUT);  
33     pinMode(green2, OUTPUT);  
34     // pinMode(blue2, OUTPUT);  
35     pinMode(red3, OUTPUT);  
36     pinMode(green3, OUTPUT);  
37     // pinMode(blue3, OUTPUT);  
38  
39     // imposto il pin del piezo in modalità OUTPUT  
40     pinMode(piezo, OUTPUT);  
41  
42     switchOffAllLEDs();  
43     digitalWrite(piezo, LOW);  
44  
45     randomSeed(analogRead(0));  
46  
47     while (!Serial) {  
48         ; // Attendi che la seriale sia pronta  
49     }  
50  
51     Serial.println("ARDUINO: Pronto per gioco Wordle");  
52     Serial.println("ARDUINO: In attesa di GAME_START...");  
53 }
```

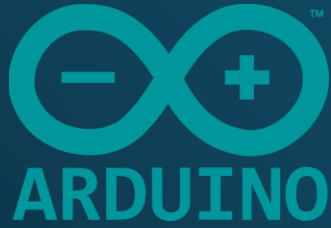



Il codice dell'Arduino

Nel codice della scheda Arduino UNO sono presenti sette funzioni:

- void setup()
- void loop()

```
55  void loop() {  
56      if (Serial.available()) {  
57          String command = Serial.readStringUntil('\n');  
58          command.trim();  
59          processCommand(command);  
60      }  
61  }
```

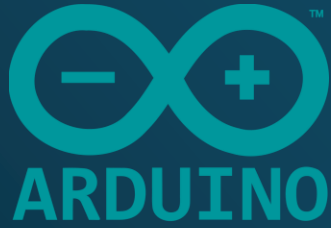


Il codice dell'Arduino

Nel codice della scheda Arduino UNO sono presenti sette funzioni:

- void setup()
- void loop()
- void processCommand(String command)

```
63  void processCommand(String command) {  
64      if (command == "GAME_START") {  
65          startNewGame();  
66      }  
67      else if (command.startsWith("TRY:")) {  
68          if (!gameActive) {  
69              Serial.println("ERROR: Gioco non attivo");  
70              return;  
71          }  
72      }  
73      String attempt = command.substring(4);  
74      processAttempt(attempt);  
75  }  
76 }
```

Il codice dell'Arduino

Nel codice della scheda Arduino UNO sono presenti sette funzioni:

- void setup()
- void loop()
- void processCommand(String command)
- void startNewGame()

```
78 void startNewGame() {  
79     switchOffAllLEDs();  
80  
81     // Genera una soluzione casuale di 3 cifre  
82     soluzione = "";  
83     for (int i = 0; i < NUM_DIGITS; i++)  
84         soluzione += String(random(0, 10));  
85  
86     for (int i = 0; i < BASE; i++)  
87         countersSol[i] = 0;  
88  
89     for (int i = 0; i < soluzione.length(); i++)  
90         // conta quante occorrenze di ogni cifra ci sono nella soluzione  
91         countersSol[(int)(soluzione.charAt(i)) - 48]++;  
92  
93     currentAttemptCount = 0;  
94     gameActive = true;  
95  
96     Serial.println("ARDUINO: Nuova partita - Soluzione: " + soluzione);  
97     Serial.println("GAME_START");  
98 }
```

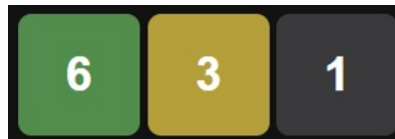


Il codice dell'Arduino

Nel codice della scheda Arduino UNO si

```
100 void processAttempt(String attempt) {
101     for (int i = 0; i < BASE; i++)
102         valutati[i] = 0;
103     currentAttemptCount++;
104
105     // Calcola i colori per ogni cifra
106     String colors[NUM_DIGITS];
107
108     // Prima passata: cifre corrette nella posizione giusta (right)
109     for (int i = 0; i < NUM_DIGITS; i++) {
110         if (attempt.charAt(i) == soluzione.charAt(i)) {
111             cInput = (int)(attempt.charAt(i)) - 48;
112             colors[i] = "right";
113             valutati[cInput]++; // incremento il numero di cInput valutati
114
115             // faccio capire al sistema quale led deve accendere
116             switch (i) {
117                 case 0:
118                     redPin = red1;
119                     greenPin = green1;
120                     break;
121                 case 1:
122                     redPin = red2;
123                     greenPin = green2;
124                     break;
125                 case 2:
126                     redPin = red3;
127                     greenPin = green3;
128                     break;
129             }
130
131             led(redPin, greenPin, 0, 255);
132             // accendo il led di verde
133         }
134     }
```

attempt:

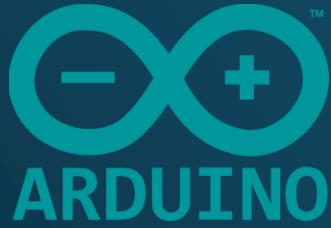


comm

empt)

```
136 // Seconda passata: cifre corrette ma in posizione sbagliata (wrong) o assenti (empty)
137 for (int i = 0; i < NUM_DIGITS; i++) {
138     if (colors[i] == "right") continue; // Salta quelle già segnate come right
139
140     cInput = (int)(attempt.charAt(i)) - 48;
141
142     // faccio capire al sistema quale led deve accendere
143     switch (i) {
144         case 0:
145             redPin = red1;
146             greenPin = green1;
147             break;
148         case 1:
149             redPin = red2;
150             greenPin = green2;
151             break;
152         case 2:
153             redPin = red3;
154             greenPin = green3;
155             break;
156     }
157
158     if (valutati[cInput] < countersSol[cInput]) {
159         colors[i] = "wrong";
160         led(redPin, greenPin, 255, 127);
161         // accendo il led di arancione
162     }
163     else {
164         // se l'i-esimo carattere non è presente altre volte nella soluzione
165         led(redPin, greenPin, 255, 0);
166         // accendo il led di rosso
167         colors[i] = "empty";
168     }
169     valutati[cInput]++;
170 }
```

```
172 // Costruisce la stringa di colori
173 String colorString = "COLORS:";
174 for (int i = 0; i < NUM_DIGITS; i++) {
175     if (i > 0) colorString += ",";
176     colorString += colors[i];
177 }
178
179 Serial.println(colorString);
180
181 // Controlla vittoria/sconfitta
182 if (soluzione == attempt) {
183     Serial.println("GAME_OVER:WIN");
184     gameActive = false;
185     Serial.println("ARDUINO: VITTORIA!");
186     for (int i = 0; i < 3; i++) {
187         tone(piezo, 1000, 100);
188         delay(250);
189     }
190     noTone(piezo);
191 }
192 else if (currentAttemptCount >= MAX_ATTEMPTS) {
193     Serial.println("GAME_OVER:LOSE");
194     gameActive = false;
195     Serial.println("ARDUINO: SCONFITTA - Soluzione: " + soluzione);
196 }
197 }
```

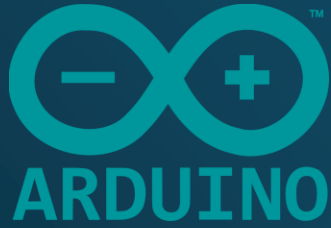


Il codice dell'Arduino

Nel codice della scheda Arduino UNO sono presenti sette funzioni:

- void setup()
- void loop()
- void processCommand(String command)
- void startNewGame()
- void processAttempt(String attempt)
- void led(int redPin, int greenPin, int red, int green)

```
205  void led(int redPin, int greenPin, int red, int green) {  
206      // limita red e green all'intervallo [0, 255]  
207      red = constrain(red, 0, 255);  
208      green = constrain(green, 0, 255);  
209  
210      // illumina i led delle quantità passate  
211      digitalWrite(redPin, red);  
212      digitalWrite(greenPin, green);  
213  }
```

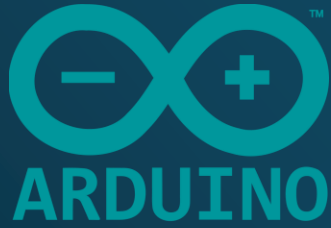


Il codice dell'Arduino

Nel codice della scheda Arduino UNO sono presenti sette funzioni:

- void setup()
- void loop()
- void processCommand(String command)
- void startNewGame()
- void processAttempt(String attempt)
- void led(int redPin, int greenPin, int red, int green)
- void switchOffAllLEDs()

```
218  ▼  void switchOffAllLEDs() {  
219      led(red1, green1, 0, 0);  
220      led(red2, green2, 255, 255);  
221      led(red3, green3, 0, 0);  
222  }
```



Il codice dell'ESP32

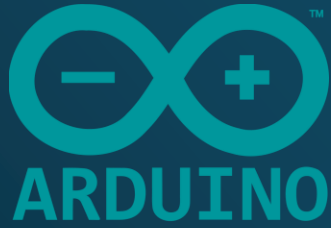
Nel codice della scheda ESP32 sono presenti tre funzioni:

- void setup()

```
434 // Setup WebSocket
435 websocket.begin();
436 // Gestione eventi WebSocket
437 websocket.onEvent([(uint8_t num, WStype_t type, uint8_t * payload, size_t length) {
438     switch(type) {
439         case WStype_DISCONNECTED:
440             Serial.printf("[%u] Disconnesso\n", num);
441             break;
442         case WStype_CONNECTED:
443             {
444                 IPAddress ip = websocket.remoteIP(num);
445                 Serial.printf("[%u] Client connesso: %s\n", num, ip.toString().c_str());
446                 websocket.sendTXT(num, "Benvenuto! Inserisci 3 cifre e premi INVIO");
447             }
448             // Avvia il gioco quando si connette il primo client
449             if (!gameActive) {
450                 gameActive = true;
451                 Serial2.println("GAME_START");
452                 Serial.println("Inviato GAME_START ad Arduino");
453             }
454             break;
455         // Gestione messaggi in arrivo
456         case WStype_TEXT:
457             {
458                 String message = String((char*)payload);
459                 Serial.printf("[%u] Ricevuto: %s\n", num, message.c_str());
460
461                 if (message.startsWith("TRY:")) {
462                     String attempt = message.substring(4);
463                     Serial2.println("TRY:" + attempt);
464                     Serial.println("Inviato ad Arduino: TRY:" + attempt);
465                 }
466                 else if (message == "GAME_START") {
467                     Serial2.println("GAME_START");
468                     Serial.println("Inviato GAME_START ad Arduino");
469                     gameActive = true;
470                 }
471             }
472             break;
473     }
474 }]);
```

```
1 #include <Arduino.h>
2 #include <Wifi.h>
3 #include <WebServer.h>
4 #include <WebSocketsServer.h>
5
6 // Configurazione Access Point
7 const char* ssid = "ESP32-Mordle";
8 const char* password = "12345678";
9
10 WebServer server(80);
11 WebSocketsServer websocket(81);
12
13 String serialBuffer = "";
14 bool gameActive = false;
15
16 // HTML page
17 const char* htmlPage = R"rawliteral(
18 <!DOCTYPE html>
```

```
409 void setup() {
410     Serial.begin(115200);
411     Serial2.begin(9600); // Comunicazione con Arduino
412
413     delay(1000);
414
415     Serial.println("=== AWVIO ESP32 ACCESS POINT ===");
416
417     // Crea Access Point
418     Wifi.softAP(ssid, password);
419     delay(2000);
420     // Ottieni e stampa l'IP dell'AP
421     IPAddress myIP = Wifi.softAPIP();
422     Serial.println("Access Point creato!");
423     Serial.printf("SSID: %s\n", ssid);
424     Serial.printf("Password: %s\n", password);
425     Serial.printf("IP: %s\n", myIP.toString().c_str());
426
427     // Setup server HTTP
428     server.on("/", HTTP_GET, []() {
429         server.send(200, "text/html", htmlPage);
430     });
431
432     server.begin();
```

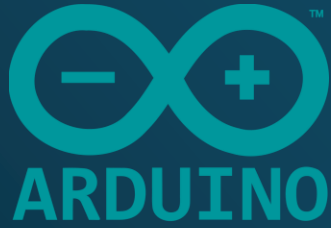


Il codice dell'ESP32

Nel codice della scheda ESP32 sono presenti tre funzioni:

- void setup()
- void processArduinoMessage(String message)

```
480  void processArduinoMessage(String message) {  
481      Serial.println("Ricevuto da Arduino: " + message);  
482  
483      if (message.startsWith("COLORS:")) {  
484          websocket.broadcastTXT(message);  
485      }  
486      else if (message.startsWith("GAME_OVER:")) {  
487          websocket.broadcastTXT(message);  
488          gameActive = false;  
489      }  
490      else if (message == "GAME_START") {  
491          websocket.broadcastTXT("GAME_START");  
492          gameActive = true;  
493      }  
494  }
```

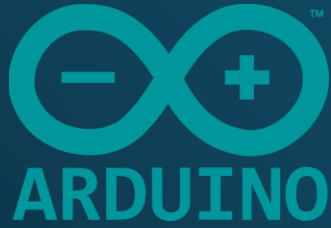



Il codice dell'ESP32

Nel codice della scheda ESP32 sono presenti tre funzioni:

- void setup()
- void processArduinoMessage(String message)
- void loop()

```
496  void loop() {  
497      server.handleClient();  
498      websocket.loop();  
499  
500      // Leggi dalla seriale Arduino  
501      while (Serial2.available()) {  
502          char c = Serial2.read();  
503          if (c == '\n') {  
504              processArduinoMessage(serialBuffer);  
505              serialBuffer = "";  
506          } else {  
507              serialBuffer += c;  
508          }  
509      }  
510  }
```



Il codice Javascript

Di seguito il punto principale del codice Javascript, che riceve i messaggi che indicano lo stato del gioco ed esegue la relativa funzione.

```
201 // Gestione messaggi in arrivo
202 websocket.onmessage = function(event) {
203     console.log("Ricevuto:", event.data); // Debug
204
205     // Gestione messaggi specifici
206     if (event.data.startsWith("COLORS:")) { //Gestione messaggio COLORS per colorare le caselle
207         // Trim globale per rimuovere \r\n finali
208         const colorString = event.data.substring(7).trim();
209         // rimuove spazi e CR/LF da ogni elemento
210         const colors = colorString.split(',').map(c => c.trim());
211         console.log("Parsed COLORS:", colors);
212         applyColorsToCurrentRow(colors);
213     }
214     else if (event.data.startsWith("GAME_OVER:")) { // Gestione messaggio GAME_OVER per capire vittoria/sconfitta
215         const result = event.data.substring(10).trim();
216         handleGameOver(result);
217     }
218     else if (event.data === "GAME_START") { // Gestione inizio nuova partita per resettare lo stato e sincronizzare esp32 e arduino
219         resetGame();
220         updateStatus("Nuova partita - Inserisci " + numeri + " cifre");
221     }
222
223     else {
224         updateStatus(event.data);
225     }
226 };
```



Repository

Di seguito il link per accedere alla [repository](#) su GitHub contenente tutti i codici presentati.

